

The Claude Code Self-Feeding Wiki Playbook

Build a Karpathy-style second brain in Claude Code that feeds itself context from your apps.

WHAT YOU'LL LEARN

- ▶ Grab the Skill First
- ▶ Your First Wiki in 5 Minutes
- ▶ What a Karpathy Wiki Actually Is
- ▶ How Raw and Wiki Work
- ▶ Context Farming — The Power Move
- ▶ Technical Setup — What Actually Gets Installed

01

Grab the Skill First

DOWNLOAD THE `BUILD-WIKI` SKILL

https://drive.google.com/file/d/1yHG2JvkY_uWzTcN_2PH3VW7ebhcyHjls/view?usp=drivesdk

Unzip into your repo's `.claude/skills/` directory and you're ready. Everything in this guide assumes you have the skill installed.

02

Your First Wiki in 5 Minutes

With the `build-wiki` skill installed, open a private repo in Claude Code and type `build wiki`. That's the whole quick win. The skill interviews you for about 3 minutes (what topic, what sources, how deep), then scaffolds a working wiki with git hooks, auto-commit, auto-push, and one farmer ready to run. It will even manage the push to a private repo in Github.

You'll end with two folders and a config file:

```
raw/           # source material drops here (transcripts, articles, messages)
wiki/          # LLM-maintained pages with cross-references
CLAUDE.md      # your bespoke schema — the LLM reads this before every ingest
```

Run `/build-wiki farm` and the farmer pulls your first batch of sources. Run it again and the wiki ingests them into linked pages. That's the loop.

QUICK WIN

If you already have a private repo open in Claude Code, you can have a working self-feeding wiki in under 10 minutes. Don't pre-plan the schema. The skill co-designs it with you.

03

What a Karpathy Wiki Actually Is

A Karpathy wiki is just markdown files in a git repo. Two folders, one config, no infrastructure.

The LLM reads your raw sources and writes human-readable wiki pages with `[[wikilinks]]` between them. When you ask a question, the LLM navigates the wiki the same way you would — read the index, follow a backlink, open a page, click through to the sources. No embeddings, no vector database, no retrieval ranking to tune.

The whole thing opens in Obsidian. You can see the graph, read every page, spot what's missing, and edit the schema by hand. That readability is the point. A second brain you can't read isn't a second brain — it's a black box you hope is right.

Property	Why it matters
Just markdown	Portable. No lock-in. Works with any editor, any tool that reads files.
Git-tracked	Full history. Every page change is a commit. Rollback is free.
Human-readable pages	You can open it and read it. Spot errors. Trust the output.
Wikilinks, not embeddings	Connections are explicit. You can see why two pages are linked.
LLM-maintained	Pages update themselves as new sources come in.
Co-designed schema	<code>CLAUDE.md</code> defines your wiki's structure. You decide what a page is.

WHERE IT FITS

Personal knowledge bases, company knowledge, topic trackers, research notebooks. Anywhere you want the AI's "memory" to be something you can actually open and read. It's newer than RAG and hasn't been proven at org scale yet, but for a second brain that's the right shape.

04

How Raw and Wiki Work

`raw/` is immutable. Sources drop in as timestamped markdown files, each one tagged with where it came from. The wiki never writes back to raw. This gives you a full audit trail — every wiki page can trace its claims back to a source file.

`wiki/` is LLM-maintained. When new files land in raw, you run `ingest` and the LLM reads the sources, updates the relevant wiki pages, creates new ones, and links them to related pages. Each page has a `sources:` frontmatter listing which raw files informed it.

The magic is in `CLAUDE.md`. That file defines your schema — what counts as a page, what counts as a link, how to classify sources, how to deduplicate. Karpathy left the schema deliberately vague because it's supposed to be co-designed with the LLM for your specific topic. A wiki for AI research looks nothing like a wiki for customer renewals.

TIP

Don't template your `CLAUDE.md`. Let the skill interview you. It asks what you care about, what you want the wiki to surface, and writes the schema from your answers. That bespoke fit is what makes the wiki feel like yours.

05

Context Farming — The Power Move

A wiki is only as current as what you feed it. Manually clipping articles and pasting transcripts is what kills every second brain. You do it for two weeks, then life happens, and the wiki stops knowing anything new.

Context farming fixes that. A farmer is a Claude Code subagent — one markdown file in `.claude/agents/` — that connects to a source through MCP and does the fetching for you. Point a farmer at a YouTube channel, a Slack workspace, a Fireflies account, a Gmail label, an arXiv category. It runs on a schedule, pulls anything from the last 24 hours, drops timestamped files into `raw/`, and commits.

The key insight from Karpathy: LLMs have bad taste. You don't want an AI scraping the open internet. But you can trust an AI to *fetch from sources you already chose*. Selecting sources is curation — that's the human job. Fetching from them is a scheduled task — that's the farmer.

Layer	Who does it	How often
Curate sources	You (once)	When you set up the farmer
Fetch new material	Farmer	Daily
Ingest into wiki	LLM	After each farm run
Read the wiki	You or an agent	Whenever

Every farmer follows the same pattern: read-only MCP tools, 24-hour window per run, append-only to `raw/`, auto-commit via hook. The skill generates them for you — you don't write the subagent file by hand.

CALLOUT

A farmer is not just a scraper. Scrapers hit the open internet and dump noise. Farmers hit sources you trust, one at a time, through MCP connectors that require your auth. Your taste stays in charge.

06

Technical Setup — What Actually Gets Installed

When you run `build wiki`, the skill runs two scripts and one interview. Here's what it installs.

Folders:

```
raw/           # source drops
wiki/          # LLM-maintained pages
.claude/
  agents/
    youtube-farmer.md  # one file per farmer
  hooks/
    auto-commit.sh     # commits after every file write
    session-sync.sh    # git pull on session start
    subagent-push.sh   # git push after each farmer run
  settings.json        # wires hooks + allowlists read-only MCP tools
CLAUDE.md       # your wiki schema
.gitignore
```

PRIVACY GATE

The init script checks `gh repo view --json visibility` before it writes anything. Public repo, it stops. The wiki contains personal context — it belongs in a private repo.

Hooks (critical):

Hook	Fires on	What it does
<code>auto-commit.sh</code>	PostToolUse (any write)	Stages and commits the new file
<code>subagent-push.sh</code>	SubagentStop (any <code>*-farmer</code> finishes)	Pushes to GitHub
<code>session-sync.sh</code>	SessionStart	Pulls latest before you work

The hook naming matters. Farmer subagent names **must** end in `-farmer` — that's what the `SubagentStop` hook keys off of. The skill enforces this.

PERMISSIONS

Farmers get read-only MCP tools only. The skill never adds `send`, `write`, or `delete` tools to the allowlist. A Slack farmer can read channels but can't post. A Gmail farmer can read email but can't send.

07

Scheduling — Cloud, Desktop, or Manual

Once the wiki works, you need to pick where farmers run. There are three modes and the right one depends on what your farmer uses.

Mode	Where it runs	Laptop needed	Works with
Cloud (Managed Agents)	Anthropic infra	No	Remote MCP connectors only (Slack, Gmail, Fireflies, Notion)
Desktop cron (Claude Desktop)	Your machine	Yes, awake	Anything — local tools, file reads, remote MCPs

Terminal loop (Claude Code <code>/loop</code>)	Current session	Yes, session open	Short bursts during active work
Manual	You trigger it	Yes	Any farmer

The GitHub piece is mandatory for cloud. Cloud scheduling clones your private repo, runs the farmer, commits new raw files, and pushes. If there's no remote, cloud can't sync. The skill gates on this — if Phase 2 detected no remote and you pick cloud in Phase 6, it stops and offers to run `gh repo create` for you.

Cloud requirements:

1. Private GitHub repo with the wiki committed
2. MCP connectors attached via claude.ai/settings/connectors (Slack, Gmail, Fireflies, etc.)
3. Farmer uses only remote MCP tools — no local binaries like `yt-dlp`

Desktop cron requirements:

1. Claude Desktop installed with your MCP servers configured
2. Any local CLIs the farmer needs (`yt-dlp`, custom scripts)
3. Laptop awake at scheduled time (or a small cloud VM running the desktop app)

TERMINAL LOOP (``/LOOP``)

Good for rapid iteration while building the farmer. Type `/loop 30m /build-wiki farm slack` and it re-runs every 30 minutes for as long as your session is open.

RULE OF THUMB

If every tool your farmer calls is a remote MCP connector, pick cloud. If it shells out to a local tool, pick desktop. If you're testing, pick manual or `/loop`.

08

The GitHub Sync Backbone

One repo is the backbone of the whole system. Farmers write to it. Your Claude Code sessions pull from it. Obsidian syncs through it. Cloud scheduled runs clone it, run, commit, push. Your teammates clone it and

get the same wiki you have.

Three paths all converge on that one repo:

```

      GitHub (private repo)
      [ ] [ ] [ ]
push from [ ] [ ] pull [ ] sync
      [ ] [ ] [ ]
      Farmers Claude Obsidian
      (scheduled) Code (git plugin)

```

Every farmer commit tags its source in the message: `wiki: farm + ingest youtube (5 new)`. Every manual ingest shows up in the log too. Open the repo on GitHub and you can see exactly what the wiki learned today, yesterday, last week.

WHY THIS MATTERS FOR TEAMS

A single team wiki with farmers across Slack, Fireflies, and Gmail means every team member wakes up to the same context. You close a deal and the relevant pages update overnight. Your onboarding new hire clones the repo and has six months of company context on day one.

09

Setting Up Your Second Brain — What to Think About

The skill does the technical work. You still have to decide what this wiki is *for*. These are the five questions worth sitting with before you run `build wiki`.

1. **What's the scope of this wiki?** It can be narrow (one topic, one research field) or broad (your whole org's Slack, every meeting, every customer). Both work. The caution is going too big on day one — a wiki tracking "everything" tends to produce muddy pages and fuzzy links. Start with the scope you can actually feed well. You can always widen it later, or spin up a second wiki for a different scope.
1. **Who asks it questions?** You? An AI agent that runs your day? Your team? The answer shapes the schema. A personal research wiki can be messy. A team wiki needs consistent page titles and strict source attribution.

1. **What are the two or three sources that actually feed this topic?** Pick the ones you already trust. Your favorite YouTube channels. Your team's Slack. A specific newsletter. Don't try to hook up ten sources on day one.
1. **What does a page look like?** Concept pages, person pages, project pages, event pages — which of these apply? The skill's interview walks you through this, but come in with a rough answer.
1. **What's the first thing you'd want to ask it in 30 days?** Work backwards from that. If the answer is "what are my customers saying about feature X?", your sources are Slack and Fireflies and your pages are per-customer and per-feature. The question picks the schema.

THE HONEST TIP

Start smaller than you think. One topic, one source, one farmer, one daily schedule. Let it run for a week before you add anything. Most wikis die from day-one over-ambition. A tiny working wiki beats a huge broken one every time.

10

The Compounding Effect

Here's what changes once this is running. You stop being the bottleneck. Your wiki learns while you sleep. Every morning, there's new context that wasn't there yesterday — new transcripts, new Slack threads, new papers — already ingested, already linked, already queryable.

And because every farmer writes to the same vault, the context compounds across sources. Slack caught a P1 ticket. Fireflies caught the board-review context that explains why the ticket matters. The wiki connects them. You didn't attend either conversation. You still know what happened.

That's the payoff. Not "I built an AI wiki." The payoff is: you walked into a meeting fully briefed without reading a message. Or: your next Claude Code session starts with 30 days of context from your past sessions, because a session farmer captured it. Or: your team onboards in a day because the wiki has six months of decisions linked and attributed.

A second brain that doesn't feed itself is a diary. A second brain that farms its own context is an unfair advantage.

WANT HELP BUILDING THIS FOR YOUR BUSINESS?

You bring the problem. I'll tell you exactly what to build. 30 minutes, your specific situation, no generic templates.

cal.com/bradley-bonanno/ai-strategy-call